Échéance : vendredi 12 juin 2026, 20:00

Nous vous avons exposé précédemment les attentes minimales concernant ce projet.

Voici maintenant **quelques pistes à explorer** pour l'améliorer.

Important : comme précédemment, le rendu se fera sur Gitlab.

Vous travaillerez sur une branche nommée **v1.0** (`git checkout - b v1.0`).
Le commit correspondant au rendu devra être tagué **v0.1** (`git tag v0.1 -m "version 0.1"`).

Attention à scrupuleusement respecter les noms de branche et de commit, sinon votre rendu ne sera pas détecté, ce qui serait particulièrement pénalisant pour le rendu final !

Contrairement aux premières étapes, **vous n'êtes plus guidés ici**.
Les objectifs proposés ici sont volontairement ambitieux, pour vous permettre de déployer votre créativité et vos talents.
Vous avez le droit à l'échec, ce n'est pas dramatique ! En ce cas, il faudra préciser en quoi cela n'a pas fonctionné comme vous l'espériez, expliquer pourquoi et donner autant que possible quelques pistes à envisager pour résoudre les problèmes rencontrés.

Dans tous les cas (réussite ou échec des améliorations envisagées), il vous faudra **expliquer votre démarche**, **évaluer la pertinence de vos choix** et **détailler vos résultats**.
Bref, prenez du recul et faites preuve d'esprit critique !

Vous pouvez aussi proposer votre propre amélioration, à condition de me l'avoir fait valider avant. Il faut que votre proposition améliore la recherche et présente un intérêt algorithmique. En particulier, il ne faut pas que ce soit une amélioration seulement d'ordre cosmétique.

Attention, vous n'aurez très probablement pas le temps de partir dans toutes les directions !

Choisissez **une** piste d'amélioration qui vous semble intéressante, et **concentrez-vous dessus**.

En parallèle, un ou deux étudiants peuvent continuer à travailler sur l'interface web (typiquement, des étudiants qui auraient plus de difficultés).

Indexation à froid

Pour l'instant, vous avez utilisé des fichiers d'index déjà fournis.

Autrement dit, vous n'avez fait que la 2e moitié du travail : exploiter les résultats de l'indexation pour fournir des résultats en temps réel.

Si vous voulez pouvoir effectuer des recherches sur un autre site que Vikidia.fr, il vous faut donc maîtriser aussi la phase amont d'indexation.

Cela suppose :

- de savoir récupérer une page web d'un site (effectuer une requête GET)
- de parser la page pour récupérer le texte de la page (sans les balises notamment)
- de lemmatiser son contenu et de générer les fichiers d'index

Dans un premier temps, vous pouvez vous contenter de fournir manuellement une liste d'URL de pages à indexer.

Dans un second temps, l'objectif serait d'arriver à parcourir le site en suivant les liens hypertextes des pages pour construire la liste de toutes les URL du site, et ne pas avoir à les fournir manuellement. Attention à ne pas tourner en rond !

Remarques :

- L'indexation n'a pas à être très rapide, puisqu'on l'effectue à froid, et non en temps réel. Prévoyez idéalement de pouvoir l'interrompre et la reprendre.
- L'indexation et la recherche sont deux programmes indépendants (leur seul lien étant le format des fichiers d'index). Vous pouvez éventuellement programmer le processus d'indexation dans

un autre langage de programmation si vous le souhaitez.
- Lors de l'indexation, soyez respectueux des serveurs (par exemple, prévoyez une seconde de délai entre deux requêtes), pour ne pas risquer de vous faire bannir (IP blacklistée).

Autocorrection

Il est fréquent de taper un mot incorrectement, soit par inadvertance, soit par ce qu'on ne connaît pas son orthographe.

Non corrigées, ces erreurs vont évidemment fortement impacter le nombre et la qualités des résultats de la recherche.

Plusieurs approches peuvent être utilisées pour renvoyer malgré tout des résultats pertinents.

- On peut utiliser un dictionnaire des erreurs les plus prévisibles, notamment concernant les fautes d'orthographe. Par exemple, "parmis" -> "parmi", ou "emener" -> "emmener".
- On peut aussi chercher les mots existants les plus proches.
- On peut utiliser des approches probabilistes (IA), mais cela nécessite l'analyse de très grandes quantités de données (phase d'apprentissage).

La 2e approche est la plus pertinente en ce qui nous concerne.

Voici quelques pistes si vous souhaitez vous pencher sur la question :

- La distance de Levenshtein permet de mesurer la distance entre deux mots, mais elle est assez coûteuse à calculer.
- On peut commencer par faire une préselection de mots *assez* proches avec une distance rapide à calculer, par exemple la distance de Jaccard, et calculer la distance de Lebenshtein seulement sur les mots sélectionnés pour affiner la correction.

Améliorer l'évaluation de la pertinence des résultats

Le modèle de *sac de mots* que nous avons adopté a ses limites, d'autant plus que la page est longue.

Par exemple, si on recherche « pomme de terre », on pourrait obtenir la page d'une ville avec un paragraphe culinaire sur une spécialité à base de pommes et un autre paragraphe historique sur les terres rattachées à la ville au Moyen-Âge. Aucun rapport avec notre recherche...

Pour résoudre ce problème, il faudrait donc tenir compte de l'ordre des mots dans la page, et non seulement de l'ensemble des mots.

Cependant, l'approche par sac de mots garde un avantage certain en terme de simplicité et de vitesse de traitement.

On pourrait donc imaginer une recherche qui fait intervenir en interne deux phases :

- on préselectionne un certain nombre de pages via une recherche vectorielle comme on le fait actuellement
- sur les pages sélectionnée, on effectue une 2e analyse plus poussée en tenant compte de la disposition des mots dans la page

Cela suppose une double indexation du contenu :

- On conserve les fichiers d'index actuels, pour pouvoir effectuer rapidement la 1re phase, exactement comme on le faisait déjà jusque-là.
- On garde aussi une copie locale du contenu de toutes les pages (uniquement le texte, sans les balises HTML), pour pouvoir effectuer la 2e analyse.

Le contenu des différentes pages web vous est fourni dans l'archive ci-dessous.
Chaque fichier de l'archive contient uniquement le texte d'une page web. (Encodage : UTF-8)
Le nom du fichier est le même que celui du fichier d'index, ce qui vous permettra de faire le lien entre le fichier d'index et le contenu de la page.

Quelques idées :

- *Facile* : vous pouvez **implémenter la recherche littérale**, avec des guillemets.
  Ex : la requête « *"pommes de terre" gâteau* » signifie que le résultat de votre recherche doit contenir les mots pomme, terre et gâteau (1re phase de recherche) mais aussi l'expression exacte « pommes de terre » (2e phase de filtrage des résultats).
- *Difficile* : **évaluer la distance entre les mots cherchés dans la page**. Par exemple, « *confiture cerises* » privilégiera les pages où les mots *confiture* et *cerises* sont assez proche dans la page. Il vous faudra réfléchir à une manière d'évaluer la distance entre les mots (que faire s'il y a plus de 2 mots ? Si un mot apparaît plusieurs fois dans la page ?). Il faudra aussi décider de la manière dont vous allez tenir compte de cette distance dans le classement des résultats.
  Par ailleurs, idéalement, il faudrait travailler avec des lemmes et non des mots (donc lemmatiser le texte...)

Contenu brut des pages 7Z

Contenu de chaque page (texte au format utf-8, expurgé des balises).

17.0 Mo

◀ Version 0.5 - Interface web Aller à... Évaluation ▶